Class Project Phase II Submission

Student Name: Bhupinder Singh ASURITE ID: bsingh55 Course: SER 460 Instructor: Dr. Janaka Balasooriya

1. Document explaining and justifying the design patterns you have identified for this project. A minimum of three design patterns are required.

Design Pattern 1: Singleton Pattern

For the first design pattern, I used the Singleton Pattern in the SunDevil Connect system through the SystemConfig class. Throughout the program, this design pattern ensures there is only one instance of the configuration class. The system name, event capacity limits, and other global parameters are examples of shared configuration values that may be required by multiple application components in a system like SunDevil Connect. The Singleton pattern provides consistency throughout the application by centralizing these settings in a single object rather than permitting the creation of many configuration objects.

Additionally, by reducing duplication and conflicting configuration data, this pattern enhances the design. One component of the system might use out-of-date or inconsistent settings compared to another, for instance, if different controllers or services were permitted to create distinct configuration objects. By using a singleton, you can ensure that all application components that rely on configuration data get it from the same source. EventController depends on SystemConfig in the class diagram, demonstrating how shared system settings can be used by event-related logic without the need to create additional configuration objects.

Finally, because SunDevil Connect is built as a centralized platform for event discovery, registration, club management, and administration, this paradigm works well with the system. Centralized configuration is advantageous for a centralized system. As the system expands in subsequent construction stages, the Singleton design facilitates maintainability, consistency, and simpler long-term refinement.

Design Pattern 2: Observer Pattern

For the second Design Pattern, I used the Observer Pattern in the SunDevil Connect design to support notifications for club announcements and updates. This pattern appears in the class diagram through NotificationObserver, StudentObserver, NotificationSubject, and ClubNotificationService. The purpose of this pattern is to allow one object, called the subject, to notify a group of dependent objects, called observers, whenever an important change occurs. In this project, when a club posts a new announcement or update, subscribed students should be notified without tightly coupling the club logic to student-specific notification behavior.

Additionally, by developing a flexible link between clubs and students, this pattern enhances the system design. ClubNotificationService maintains the observer list and transmits updates via the observer interface, rather than the club directly managing notification logic for every student. NotificationObserver is implemented by StudentObserver, enabling independent handling of student-related notification behavior. This lessens the tight connection and facilitates the design's expansion. For instance, without changing the club classes, the system might eventually support dashboard alerts, mobile push notifications, or email notifications.

At last, the Observer pattern fits SunDevil Connect especially well because one of the project's goals is to improve communication between club leaders and students. The requirements specifically mention club pages, announcements, updates, and member engagement. A notification-based design supports those goals in a realistic and scalable way. It also improves maintainability, since new observer types can be added later with minimal changes to the core subject class.

Design Pattern 3: Factory Method Pattern

For the Third Design Pattern, I used the **Factory Method Pattern** in the SunDevil Connect system through the UserFactory class. This pattern is responsible for creating user objects such as Student, ClubLeader, and Admin. Since all of these classes inherit from User, the factory provides a centralized way to create different user types without requiring the rest of the system to directly instantiate each subclass. This improves organization and keeps the object-creation logic in a dedicated place.

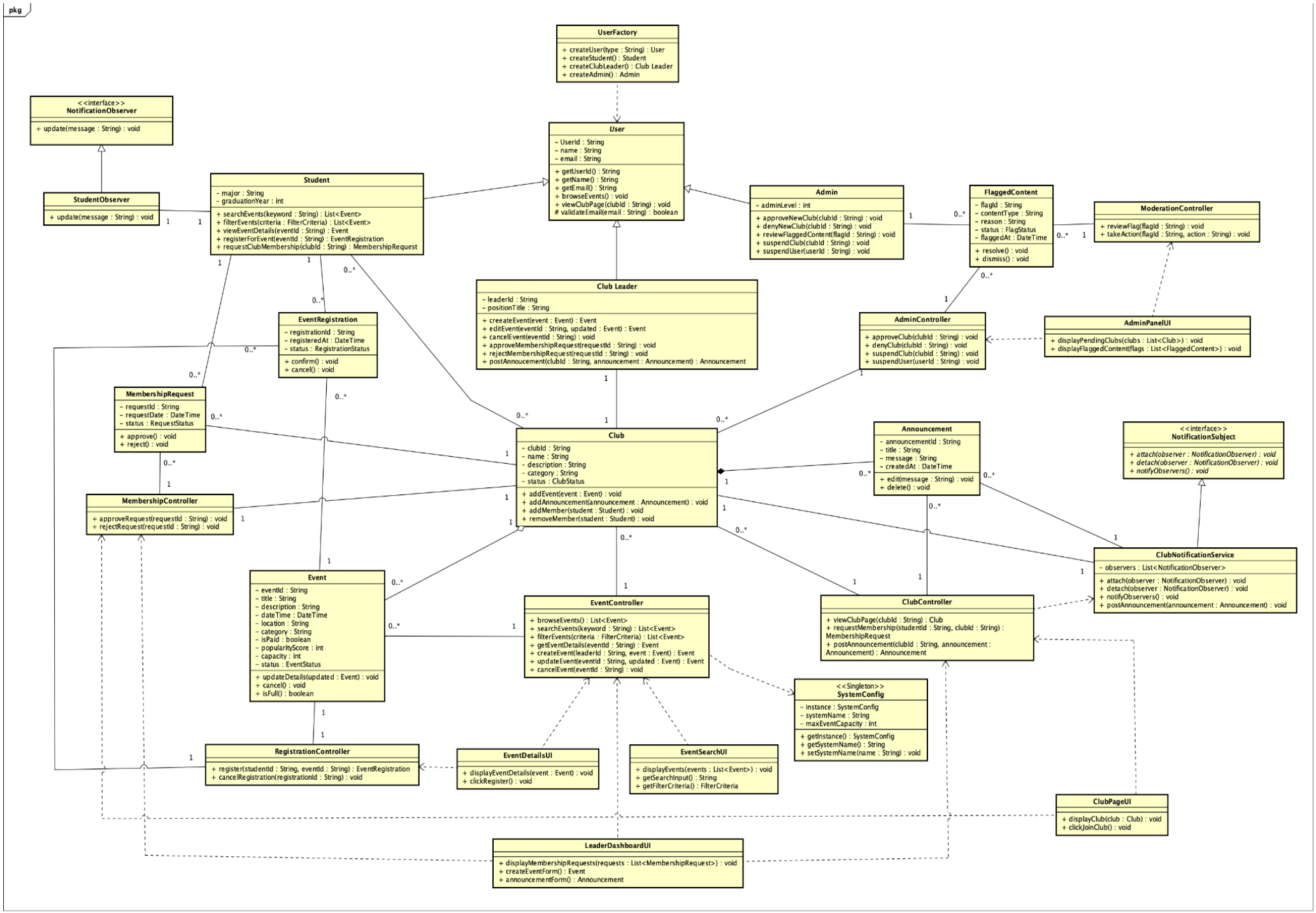
Additionally, by eliminating redundancy and facilitating system expansion, this pattern enhances the design. In the absence of a factory, several application components may use hardcoded constructors to manually build user instances. This method makes future updates more challenging by dispersing the creation logic throughout the codebase. The system can request a User object via UserFactory, and the factory will determine which subclass to construct. Instead of changing several areas of the program, the factory can be updated in a single location if a new user role is added later, such as GuestUser or SuperAdmin.

At last, this pattern is a strong fit for SunDevil Connect because the project clearly defines multiple user roles with different responsibilities; also, students discover and register for events, club leaders manage events and members, and admins oversee clubs and flagged content. Since these roles share a common base type but behave differently, a factory-based creation mechanism is both logical and scalable. The Factory Method pattern therefore improves flexibility, maintainability, and future extensibility of the system.

2. The refined class diagram clearly shows the design pattern related to classes and other classes. Astah must be used to develop the class diagram. (Copy and paste your class diagram here. Make sure to use the correct UML notations. Points will be given for the completeness and the correctness of the diagram

Class Project Phase II Submission

Student Name: Bhupinder Singh ASURITE ID: bsingh55 Course: SER 460 Instructor: Dr. Janaka Balasooriya



Class Project Phase II Submission

Student Name: Bhupinder Singh ASURITE ID: bsingh55 Course: SER 460 Instructor: Dr. Janaka Balasooriya